

Topic: College Event Management System with CI/CD

The basic idea is a website where students can see college events and register for them, while an admin can create and manage events.

1. What will our website have?

We can keep the application simple:

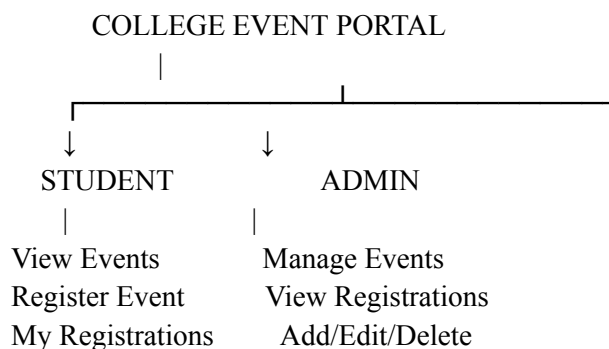
Student side

- Register/Login
- View upcoming events
- Search events
- View event details
- Register for an event
- View registered events

Admin side

- Admin login
- Add event
- Edit event
- Delete event
- View registered students

For example:



Complete project in one line

React → Node.js/Express → MySQL → Docker → Kubernetes → GitHub Actions → Container Registry → Kubernetes deployment

Project Flow

“Our project is a **College Event Management System with CI/CD**. First, we will develop the frontend using **React.js**, where students can log in, view upcoming college events, check event

details, and register for events. Administrators will have a separate interface where they can create, update, and delete events and view student registrations.

The frontend will communicate with a **Node.js and Express.js backend**. The backend will handle the application logic, such as user login, event management, and event registration. The application data will be stored in a **MySQL database**, which will contain information about users, events, and registrations.

After developing the application, we will **containerize the frontend, backend, and database using Docker**. Instead of running the application directly on the computer, each component will run inside its own container. This makes the application easier to deploy and manage.

Next, we will use **Kubernetes with Minikube** to manage these containers. Kubernetes will create and manage the required Pods and Services. It will also allow us to demonstrate Kubernetes features such as deployment, service management, scaling, and automatic recovery of containers.

For the **CI/CD part**, we will store our project code on **GitHub**. Whenever we make changes and push the code to GitHub, **GitHub Actions** will automatically run the build and testing process, create the required Docker images, and push the images to **GitHub Container Registry**. The updated images can then be deployed to our Kubernetes environment.

So, the overall flow is: **React frontend → Node.js/Express backend → MySQL database → Docker containers → Kubernetes deployment → GitHub Actions CI/CD → Docker image registry → updated Kubernetes application**. This demonstrates application development, containerization, orchestration, and automated deployment in one project.”

Modules to mention

1. Student Module:

Students can register/login, view events, see event details, register for events, and view their registrations.

2. Admin Module:

Admins can create, update, and delete events and view student registrations.

3. Event Management Module:

Handles event information such as event name, date, venue, description, and registration details.

4. Authentication Module:

Handles user login and access control between students and administrators.

5. Database Module:

MySQL stores users, events, and registrations.

6. Deployment Module:

Docker containerizes the application and Kubernetes manages the containers.

7. CI/CD Module:

GitHub Actions automatically builds, tests, creates Docker images, and deploys updated application versions.

One-line explanation for the teacher

“The architecture starts with students and administrators interacting with the React frontend, which communicates with the Node.js backend and MySQL database. The application is containerized using Docker, managed using Kubernetes, and the complete build and deployment process is automated using GitHub Actions CI/CD.”

College Event Management System with CI/CD

The **College Event Management System** is a web-based application designed to provide a centralized platform for managing and participating in college events. In a traditional college environment, event information, registrations, and participant records may be handled through different platforms or manually, which can make the process difficult to manage. Our proposed system provides a single platform where students can easily discover upcoming college events, view complete event information, and register online.

The system will have **two main types of users: students and administrators**. Students will be able to create an account, log in, browse upcoming events, view details such as event name, date, time, venue, and description, register for events, and view their registered events. Administrators will be responsible for managing the events. They can add new events, update existing event information, delete events when required, and view the list of students registered for each event.

The application will be developed using a **three-tier architecture**. The **React.js frontend** will provide the user interface and communicate with the backend. The **Node.js and Express.js backend** will handle business logic, authentication, event management, and registration requests. A **MySQL database** will store user details, event information, and registration records.

To apply DevOps and cloud-native concepts, the application components will be **containerized using Docker**. Instead of running the application directly on the system, the frontend, backend, and database will run in separate containers. **Kubernetes** will then be used for container orchestration, deployment, service management, scaling, and automatic recovery of application components.

The project will also implement a **CI/CD pipeline using GitHub Actions**. Whenever changes are pushed to the GitHub repository, the pipeline will automatically build and test the application, create updated Docker images, and push the images to a container registry. The updated application can then be deployed to the Kubernetes environment. This reduces manual deployment work and demonstrates an automated software delivery process.

The main objective of the project is therefore not only to develop an event management website, but also to demonstrate how **Docker containerization, Kubernetes orchestration, GitHub-based**

version control, and CI/CD automation can be combined to create an application that is easier to deploy, manage, update, and maintain.

Technologies

React.js | Node.js | Express.js | MySQL | Docker | Kubernetes | GitHub | GitHub Actions

Short line at the bottom of the slide

“A centralized college event platform with automated containerized deployment using Docker, Kubernetes, and CI/CD.”

CI/CD means Continuous Integration / Continuous Delivery (or Continuous Deployment).

Simple meaning

CI – Continuous Integration

Whenever a developer makes changes to the code and pushes them to **GitHub**, the system automatically **builds and tests the code** to check whether everything is working correctly.

CD – Continuous Delivery/Deployment

After the code passes the tests, the system automatically prepares and/or **deploys the updated application** to the deployment environment.

In our project

Our flow will be:

Developer → GitHub → GitHub Actions → Build & Test → Docker Image → Container Registry → Kubernetes → Updated Application

So, if we modify something in the **College Event Management System**, instead of manually building and deploying everything again, **GitHub Actions automates the process.**

Viva answer

“CI/CD is a DevOps practice used to automate the software development and deployment process. CI automatically builds and tests code changes, while CD automatically delivers or deploys the tested application. In our project, we use GitHub Actions to implement CI/CD and automate the deployment of our Dockerized application to Kubernetes.”

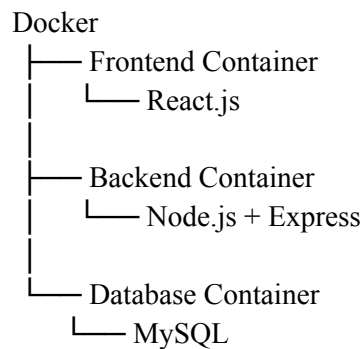
1. Where Docker helps

After we develop our application:

React Frontend → Node.js/Express Backend → MySQL Database

we use **Docker** to containerize these components.

Instead of installing and running everything directly on the computer, we package the required application and dependencies into containers.



Why Docker helps us:

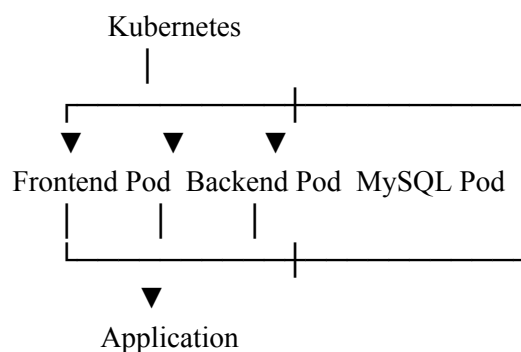
- Same application environment on different computers.
- Avoids dependency/version problems.
- Makes the application portable.
- Makes deployment easier.
- Provides separate containers for different components.

Simple viva answer:

“Docker is used in our project to containerize the frontend, backend, and database so that the application and its dependencies can run consistently in isolated environments.”

2. Where Kubernetes helps

Once we have Docker containers, **Kubernetes manages those containers.**



Kubernetes can:

- Deploy our containers.
- Start and stop containers.
- Manage Pods.
- Provide Services for communication.
- Restart a failed container.
- Run multiple replicas when required.
- Help with scaling.

For example, if our college website receives a large number of students during event registration, Kubernetes can run **multiple backend replicas** to handle the workload.

Simple viva answer:

“Kubernetes is used to manage and orchestrate our Docker containers. It handles deployment, service communication, scaling, and recovery of the application.”

Docker	Kubernetes
Creates and runs containers	Manages containers
Containerization	Container orchestration
Packages application + dependencies	Deploys and manages containers
Runs individual containers	Manages multiple Pods/containers
Makes application portable	Helps with scaling and recovery

Easy way to remember

Docker = Packs the application

Kubernetes = Manages the packed applications

And then **CI/CD connects everything**:

GitHub → GitHub Actions → Docker Image → Kubernetes → Running College Event System.

How Docker helps our project

We first develop:

React.js → Node.js/Express → MySQL

Then Docker packages these parts into containers.

For example, when we want to deploy our college website, instead of saying:

“Install Node.js, install all packages, configure MySQL, then run the application.”

we can simply run the required **Docker containers**.

So in our project:

- **React container** → runs the college event website.
- **Node.js container** → handles login, event creation, registration, etc.
- **MySQL container** → stores students, events, and registrations.

This makes our application **portable and consistent**.

How Kubernetes helps our project

After Docker creates the containers, **Kubernetes manages them**.

Suppose our college announces a popular event and **many students try to register at the same time**. Kubernetes can run multiple copies of the backend to handle the increased requests.

It also helps if something goes wrong. For example, if a backend container stops unexpectedly, Kubernetes can **restart/recreate it automatically**.

So Kubernetes gives us:

Deployment + Container Management + Scaling + Service Communication + Recovery

How they work together in OUR project

Think of it like this:

Our College Event Management System is the application.

Docker packs the application into containers.

Kubernetes manages those containers when the application is running.

CI/CD automatically updates those containers when we change the code.

Best answer for your teacher

“In our project, Docker is used to containerize the React frontend, Node.js backend, and MySQL database, which makes the application easy to package and deploy. Kubernetes is then used to manage these containers by handling deployment, service communication, scaling, and automatic recovery. CI/CD using

GitHub Actions automates the process of building and updating these Docker containers whenever we make changes to the project.”

The **React frontend** is what the student or admin sees. The **Node.js + Express backend** handles requests such as login, creating events, registering for an event, and viewing registrations. **MySQL** stores the actual data.

Docker and Kubernetes come **after we develop these application components**. They help us package, deploy, and manage the application.

Docker = Containerization

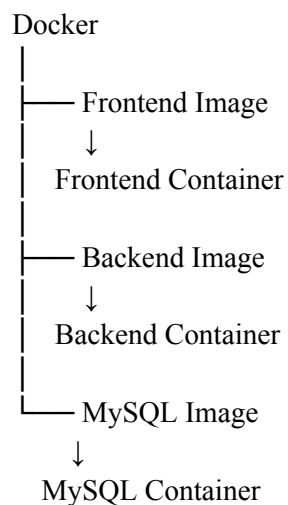
Docker allows us to package an application together with everything it needs to run.

For example, our Node.js backend requires:

- Node.js
- Express
- Other npm packages
- Our backend source code
- Configuration

Instead of installing all these things manually on another computer, we create a **Docker image** containing the required environment and application.

Then we run that image as a **container**.



Suppose a student opens:

College Event Management Website

The request first reaches our frontend.

The frontend communicates with the backend.

The backend communicates with MySQL.

Docker allows these components to run in separate isolated containers.

Docker is making sure that each part has its required environment and can run independently.

Why do we need Kubernetes if Docker already runs the containers?

This is a very important viva question.

Docker can **create and run containers**, but when we have several containers and need to manage them, Kubernetes becomes useful.

Think about our project.

Initially:

1 Frontend

1 Backend

1 Database

But suppose our college announces a very popular event.

Suddenly:

500 students

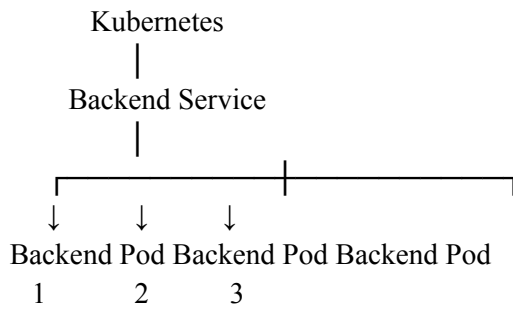


Many registration requests



Backend receives heavy traffic

Instead of having only one backend instance, Kubernetes can help us run multiple backend replicas.



The requests can be distributed among the available backend Pods.

This is where Kubernetes becomes useful.

5. How Kubernetes helps our project

Kubernetes provides **container orchestration**.

In our project, it can handle:

Deployment

It tells Kubernetes which application containers need to run.

For example:

Run the frontend application.

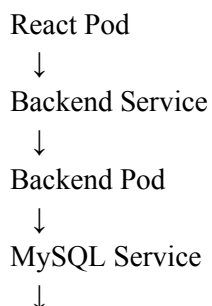
Run the backend application.

Run the database.

Service communication

Our frontend needs to communicate with our backend.

Kubernetes Services provide stable network access between application components.



MySQL Pod

So our different components can communicate with each other.

Scaling

Suppose registration traffic increases.

We can increase the number of backend replicas:

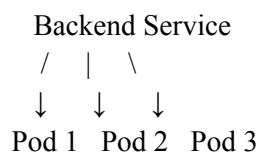
Before:

Backend Pod



Requests

After:



This allows the application to handle more requests.

Automatic recovery

Suppose one backend Pod crashes.

Without orchestration, we may need to manually restart it.

Kubernetes can detect that the required Pod is no longer running and create a replacement.

Backend Pod 1



CRASH



Kubernetes detects failure



New Backend Pod created

This is called **self-healing/automatic recovery**.

6. Where CI/CD comes into our project

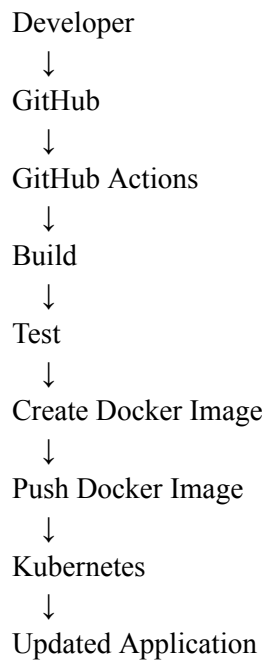
Now suppose we make a change.

For example, we add a new feature:

“Students can search events by category.”

We modify the code and push it to GitHub.

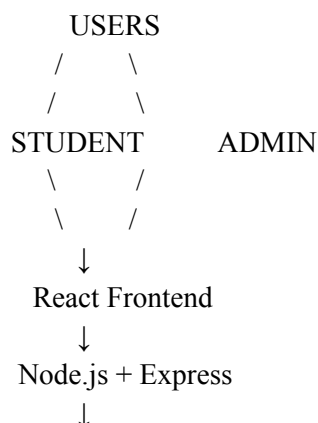
Instead of manually doing everything again, our CI/CD pipeline handles the process.

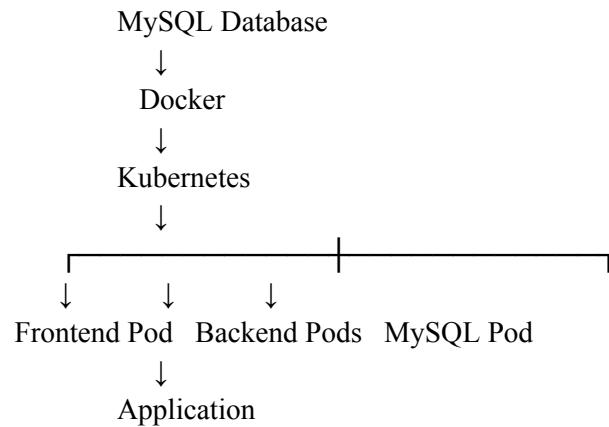


So **CI/CD connects development with Docker and Kubernetes.**

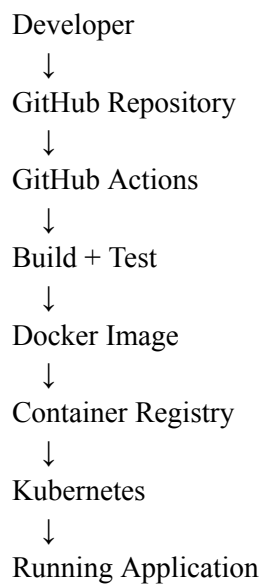
7. Complete flow of OUR project

This is the most important flow to remember for your teacher:





And the development/deployment side is:



8. Very simple difference

If your teacher asks:

“Why Docker?”

Say:

“Docker is used to containerize our application. It packages the frontend, backend and database components with their required dependencies, making the application portable, consistent and easier to deploy.”

“Why Kubernetes?”

Say:

“Kubernetes is used to manage and orchestrate our Docker containers. It handles deployment, service communication, scaling and automatic recovery of the application.”

“Why CI/CD?”

Say:

“CI/CD automates the software delivery process. In our project, GitHub Actions automatically builds and tests the code, creates the Docker image and helps deploy the updated application to Kubernetes.”

★ One sentence to remember

“Docker packages our College Event Management System, Kubernetes manages the containers, and CI/CD automates the process of updating and deploying the application.”

Full Project Summary – College Event Management System with CI/CD

Our project is a **College Event Management System with CI/CD**. The main purpose of the project is to create a centralized web-based platform through which students can easily view and register for college events, while administrators can manage all the events and registrations. Students will be able to log in, view upcoming events, check details such as event name, date, time, venue and description, register for events, and view their registrations. Administrators will be able to add new events, update or delete existing events, and view the students registered for each event.

The application will have a **frontend, backend, and database**. We will use **React.js** to develop the frontend because it provides the user interface through which students and administrators interact with the system. The **Node.js and Express.js backend** will handle the application logic and API requests, such as authentication, event management and event registration. **MySQL** will be used as the database to store user information, event details and registration records. Therefore, the basic application flow will be **React → Node.js/Express → MySQL**.

After developing the application, we will use **Docker for containerization**. Docker will package our application components and their required dependencies into containers. We can have separate containers for the React frontend, Node.js backend and MySQL database. This makes the application easier to run in different environments and avoids problems related to missing dependencies or different software versions. In simple words, **Docker packages our application so that it can run consistently**.

After Docker, we will use **Kubernetes for container orchestration**. Kubernetes will manage the Docker-based application containers. It will be responsible for deploying the application, managing Pods and Services, enabling communication between components, and providing features such as scaling and automatic recovery. For example, if the event registration system receives a large number of requests, multiple backend Pods can be used to handle the workload. If a Pod fails, Kubernetes can automatically recreate it. For our free setup, we can use **Minikube** to run Kubernetes locally.

The **CI/CD pipeline** will automate our development and deployment process. We will store our source code on **GitHub** and use **GitHub Actions** for CI/CD. Whenever we make changes to the project and push them to GitHub, GitHub Actions can automatically build and test the application. It can then create the required Docker image and push it to a container registry. The updated image can subsequently be used for the Kubernetes deployment. Therefore, instead of manually repeating the complete deployment process after every change, CI/CD automates it.

Technology	Role in our project
React.js	Creates the student/admin website
Node.js + Express	Handles backend logic and APIs
MySQL	Stores users, events and registrations
Docker	Containerizes application components
Kubernetes	Manages and orchestrates containers
GitHub	Stores and manages source code
GitHub Actions	Automates CI/CD
Container Registry	Stores Docker images
Minikube	Provides our local Kubernetes environment

Full explanation for teacher

“Our project is a College Event Management System with CI/CD. It is a web-based application where students can view and register for college events, while administrators can create, update, delete and manage events and registrations. We are using React.js for the frontend, Node.js and Express.js for the backend, and MySQL for the database. After development, we will containerize the application components using Docker. Kubernetes will then be used to deploy and manage these containers and demonstrate orchestration, scaling and automatic recovery. We will use GitHub for version control and GitHub Actions to implement CI/CD. Whenever code changes are pushed to GitHub, the pipeline will build and test the application, create the Docker image and deploy the updated application through Kubernetes. Thus, our project combines a functional college event system with Docker, Kubernetes and automated CI/CD practices.”

Viva

1. First: What is your project?

Question: What is your project?

Answer:

“Our project is a **College Event Management System with CI/CD**. It is a web-based application that provides a centralized platform for students and administrators to manage college events. Students can view events, see event details and register for events, while administrators can create, update and delete events and view student registrations. We will use Docker for containerization, Kubernetes for deployment and management, and GitHub Actions for CI/CD automation.”

2. What exactly will you develop?

Your application will have **two main users**.

Student

The student can:

Login/Register



View Events



View Event Details



Register for Event



View My Registrations

Admin

The admin can:

Admin Login



Create Event



Update Event



Delete Event



View Student Registrations

3. Which technologies will you use?

Tell her this clearly:

Component	Technology
Frontend	React.js
Backend	Node.js + Express.js
Database	MySQL
Containerization	Docker
Orchestration	Kubernetes
Local Kubernetes	Minikube
Version Control	GitHub
CI/CD	GitHub Actions
Container Registry	GitHub Container Registry (GHCR)

And importantly:

“We are using free/local tools and are not depending on AWS for this project.”

4. How will you start the project?

If she asks **“How will you do the project from beginning?”**, explain this sequence.

Step 1 – Develop the frontend

First, we will create the React.js frontend.

We will create pages such as:

- Home
 - Login/Register
 - Events
 - Event Details
 - Registration
 - My Registrations
 - Admin Dashboard
-

Step 2 – Develop the backend

Then we will create the Node.js + Express.js backend.

The backend will provide APIs for:

Authentication

Events

Registrations

Users

For example:

GET /events

POST /events

PUT /events/:id

DELETE /events/:id

POST /registrations

GET /registrations

The exact API structure can be finalized during implementation.

5. How will MySQL be used?

We will create a relational database in MySQL.

Main tables:

Users

user_id

name

email

password

role

Events

event_id

event_name

date

time

venue

description

Registrations

registration_id

user_id

event_id
registration_date

The important relationship is:

Users
↓
Registrations
↓
Events

So when a student registers for an event, the registration record connects the **student/user** with the **event**.

6. Where does Docker come in?

After developing and testing the application, we will **containerize it using Docker**.

We can create separate Docker images/containers for:

React Frontend
↓
Frontend Docker Image

Node.js Backend
↓
Backend Docker Image

MySQL
↓
Database Container/Image

If teacher asks: Why Docker?

Say:

“Docker packages the application and its dependencies into containers, which provides a consistent environment and makes deployment easier.”

Very short answer:

Docker packages our application.

7. Where does Kubernetes come in?

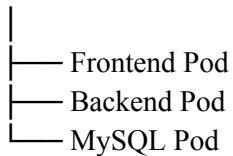
After creating the Docker containers/images, we use **Kubernetes** to manage the application containers.

Since we want a free local setup, we will use:

Minikube

The Kubernetes environment can contain:

Kubernetes Cluster



Kubernetes will help us with:

- Deployment
- Container management
- Scaling
- Self-healing
- Service discovery

If teacher asks: Why Kubernetes?

Say:

“Docker is used to create and run containers, while Kubernetes is used to manage and orchestrate those containers.”

Remember this:

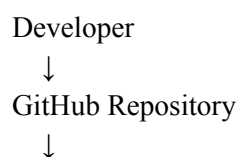
Docker = Packaging
Kubernetes = Management

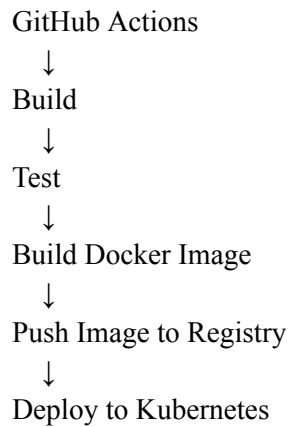
8. Where does CI/CD come in?

This is probably an **important question** because CI/CD is in your project title.

We will use **GitHub Actions**.

The flow will be:





So if we modify the application and push the code to GitHub, GitHub Actions can automatically perform the required build/test/image/deployment workflow.

If teacher asks: What is CI?

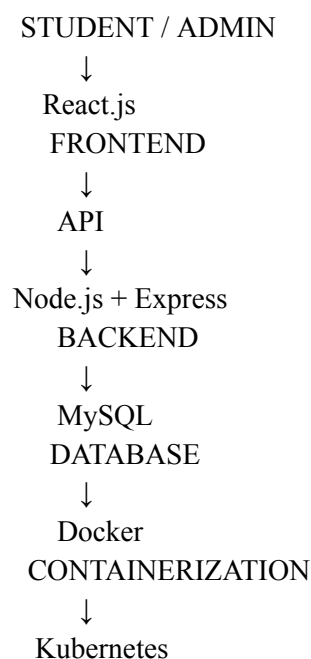
“Continuous Integration means automatically integrating code changes and running build and test processes.”

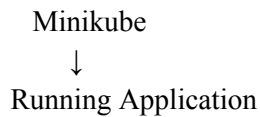
What is CD?

“Continuous Delivery or Deployment automates the process of making the tested application ready for or deploying it to the target environment.”

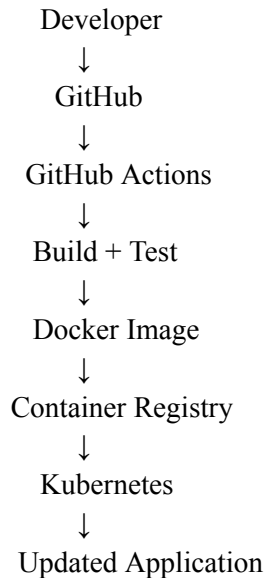
9. Complete project flow

This is the **most important thing to memorize**:





And the automation side:



10. Teacher may ask: Why did you choose this project?

Say:

“We selected this project because college events are often managed through separate or manual processes. Our system provides a centralized platform for event information and student registration. At the same time, it allows us to demonstrate the DevOps concepts covered in the subject, such as Docker, Kubernetes and CI/CD.”

11. Why React.js?

“We selected React.js because it is suitable for developing a dynamic and component-based user interface.”

Don't over-explain unless she asks.

12. Why Node.js?

“Node.js allows us to build the backend using JavaScript and is suitable for handling API requests for our web application.”

13. Why Express.js?

“Express.js is a lightweight Node.js framework that simplifies creating REST APIs and handling backend routes.”

14. Why MySQL?

“We selected MySQL because our data is structured and relational. Users, events and registrations have clear relationships, so a relational database is suitable.”

If she asks:

Why not MongoDB?

“MongoDB could also be used, but our application has structured relational data and relationships between users, registrations and events, so MySQL is more suitable and simpler for our project.”

15. Why Minikube?

“Minikube allows us to run a Kubernetes cluster locally on our computer. It is suitable for our project because we can demonstrate Kubernetes deployment without depending on a paid cloud platform.”

This is an **excellent answer** because your project needs to remain free.

16. What happens if a backend container fails?

Teacher may ask this because Kubernetes is involved.

Say:

“Kubernetes continuously monitors the desired state of the deployment. If a Pod fails, Kubernetes can recreate it according to the deployment configuration.”

That's your **self-healing** concept.

17. What happens if many students register at the same time?

Say:

“We can configure Kubernetes to run multiple backend replicas and use Kubernetes services to distribute requests. This helps the application handle increased workload.”

This demonstrates **scalability**.

18. What is the role of GitHub?

“GitHub is used to store and manage our source code and track changes made by developers.”

19. What is the role of GitHub Actions?

“GitHub Actions automates our CI/CD workflow. It can build and test the application, create Docker images, push them to the container registry and deploy the updated application to Kubernetes.”

★ Most Important Viva Question

“Explain your complete project in short.”

Memorize this paragraph:

“Our project is a College Event Management System with CI/CD. We will develop the frontend using React.js, the backend using Node.js and Express.js, and use MySQL for storing users, events and registration data. Students can view and register for events, while administrators can manage events and view registrations. After development, we will containerize the application using Docker. These containers will be deployed and managed using Kubernetes through Minikube. The source code will be maintained on GitHub, and GitHub Actions will automate the CI/CD pipeline by building, testing, creating Docker images and deploying the updated application to Kubernetes. The main purpose is to combine a functional college event system with practical DevOps concepts.”

And remember this one-line architecture:

React → Node/Express → MySQL → Docker → Kubernetes

And CI/CD:

GitHub → GitHub Actions → Docker Image → Registry → Kubernetes

How we will do the project

1. **Develop the website** using **React.js** for students and admins.
2. **Create the backend** using **Node.js + Express.js** to handle login, events and registrations.
3. **Connect MySQL** to store users, events and registration data.
4. **Test the application** locally to make sure all functions work.
5. **Create Docker images** for the frontend and backend and run the application in containers.
6. **Deploy the containers on Kubernetes (Minikube)** and create the required Pods/Services.
7. **Create a CI/CD pipeline using GitHub Actions** so that when we push code to GitHub, it automatically builds, tests, creates the Docker image and updates the Kubernetes deployment.

In one flow:

React → Node.js/Express → MySQL → Docker → Kubernetes

and

GitHub → GitHub Actions → Docker Image → Kubernetes

If teacher asks “What will you actually demonstrate?”

Say:

“We will demonstrate a working college event website, containerize it using Docker, deploy it on a local Kubernetes cluster using Minikube, and automate the build and deployment process using GitHub Actions.”

1. What is Docker?

Docker is a platform used to package and run applications inside containers.

Suppose your project needs:

- Node.js
- Express
- React
- MySQL
- Different libraries and versions

Instead of manually installing everything on every computer, Docker packages the application and its required environment into a **container**.

Simple example

Without Docker:

“Install Node.js → install packages → configure environment → run application.”

With Docker:

“Create Docker image → run container → application runs.”

Viva answer:

“Docker is a containerization platform that packages an application and its dependencies into containers so that it can run consistently in different environments.”

2. What is a Container?

A container is a lightweight, isolated environment in which an application runs.

Think of it like a **box** 📦 containing:

Application

+

Required libraries

+

Dependencies

+

Configuration

For your project, we can have:

Frontend Container

↓

React.js

Backend Container

↓

Node.js + Express

Database Container

↓

MySQL

Easy difference

Docker = Tool/platform

Container = Environment created and run using Docker

Viva answer:

“A container is an isolated and lightweight environment used to run an application along with its required dependencies.”

3. What is a Docker Image?

This is another term your teacher may ask.

A Docker image is a blueprint/template used to create a container.

Think:

Docker Image



creates



Container

For example:

Node.js Docker Image



Backend Container

Easy example

Think of a **recipe** and a **prepared dish**:

Docker Image = Recipe

Container = Prepared Dish

Viva answer:

“A Docker image is a packaged, read-only template from which containers are created.”

4. What is Kubernetes?

Kubernetes is a container orchestration platform.

This means it helps us **manage multiple containers**.

Suppose we have:

Frontend Container
Backend Container
MySQL Container

Managing all of them manually can become difficult.

Kubernetes helps with:

- Deploying containers
- Managing containers
- Scaling applications
- Restarting failed workloads
- Networking
- Load balancing

Simple idea

Docker runs containers. Kubernetes manages containers.

Viva answer:

“Kubernetes is an open-source container orchestration platform used to deploy, manage, scale and maintain containerized applications.”

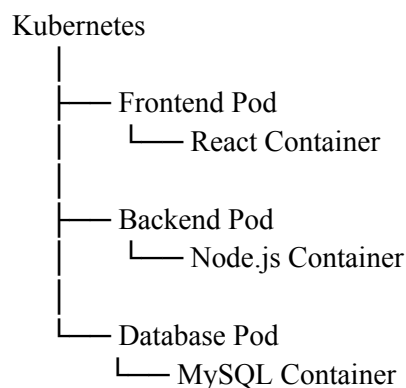
5. What is a Pod?

This is **very important** because Kubernetes doesn't directly manage individual containers in the same way you normally think about them—it manages **Pods** as its basic deployable unit.

A Pod is the smallest deployable unit in Kubernetes.

A Pod can contain **one or more containers** that share networking and storage resources.

For your project, normally we can have:



For a simple project, **one main container per Pod** is common.

Easy way to remember

Container = application environment

Pod = Kubernetes unit that runs the container(s)

Viva answer:

“A Pod is the smallest deployable unit in Kubernetes and usually contains one application container.”

6. What is Minikube?

Minikube is a tool that allows us to run Kubernetes locally on our computer.

Normally, Kubernetes can be used on a cluster of machines.

But for your college project, you don't need a cloud server.

We can install **Minikube** and create a Kubernetes cluster on our own laptop.

Your Laptop



Minikube



Kubernetes Cluster



Pods



Containers

Why are we using Minikube?

Because your project needs to be **free** and you don't want to depend on AWS.

So:

Minikube = Local Kubernetes environment

Viva answer:

“Minikube is a tool that runs a single-node Kubernetes cluster locally, mainly for learning, development and testing.”

★ Most Important Difference

Memorize this:

Term	Simple meaning
Docker	Tool for creating and running containers
Container	Isolated environment where application runs
Docker Image	Blueprint used to create a container
Pod	Smallest deployable unit in Kubernetes
Kubernetes	Manages and orchestrates containers/Pods
Minikube	Runs Kubernetes locally on your computer

Your Project Example

Suppose you have your **College Event Management System**.

Step 1 — Application

You develop:

React.js → Frontend

Node.js → Backend

MySQL → Database

Step 2 — Docker

You containerize the components:

React



Frontend Container

Node.js



Backend Container

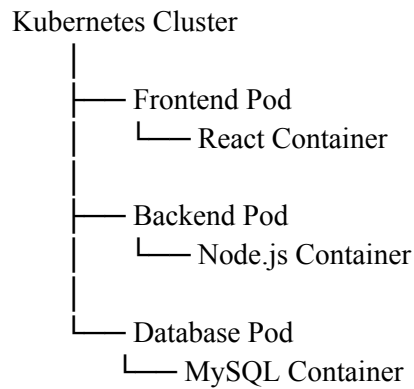
MySQL



Database Container

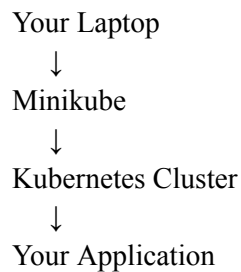
Step 3 — Kubernetes

Kubernetes runs/manages them through Pods:



Step 4 — Minikube

Instead of using a paid cloud Kubernetes service:



★ One Story to Remember Everything

Imagine you have a **college event**.

Docker = Packing boxes 📦

You put each application and its requirements into a box.

Container = The packed box

The application runs inside that box.

Kubernetes = Event manager 🧑

It manages all the boxes, makes sure they are running, and can create more when needed.

Pod = Kubernetes unit holding the box

Kubernetes organizes the running containers into Pods.

Minikube = Your practice venue 🏠

Instead of renting a huge external venue/cloud, you run Kubernetes on your own laptop for the project.

★ 5-second viva answers

Docker?

“A platform for containerizing applications.”

Container?

“An isolated environment in which an application runs.”

Pod?

“The smallest deployable unit in Kubernetes that runs one or more containers.”

Kubernetes?

“A platform for managing and orchestrating containers.”

Minikube?

“A tool for running Kubernetes locally on a computer.”

Docker vs Kubernetes?

“Docker packages and runs containers; Kubernetes manages and orchestrates them.”

Minikube vs Kubernetes?

“Kubernetes is the orchestration platform, while Minikube provides a local Kubernetes cluster for development and testing.”

If your teacher asks **"How and why are you using Docker images in your project?"**, say:

"In our College Event Management System, we use Docker images to package the frontend and backend applications along with their required dependencies. These images are then used to create containers, which are deployed as Pods in Kubernetes. Docker images provide a consistent and portable environment, while Kubernetes manages and orchestrates the containers. In our CI/CD pipeline, GitHub Actions builds the Docker image and the updated image is deployed to Kubernetes."

Remember this simple chain

Dockerfile → Docker Image → Container → Pod → Kubernetes

And:

GitHub → GitHub Actions → Docker Image → Registry → Kubernetes

FULL PRESENTATION — What You Can Actually Say

If you want to present naturally instead of sounding like you're reading, memorize this version:

“Good morning everyone. We are Group 11, and our project title is Design and Development of a Containerized College Event Management System with CI/CD.

Our project is a centralized web-based platform for managing college events and student registrations. There are two main types of users: students and administrators. Students can register and log in, view upcoming events, check event details, register for events and view their registrations. Administrators can add, update and delete events and view the students registered for each event.

We are using React.js for the frontend, Node.js and Express.js for the backend, and MySQL for the database. The React frontend communicates with the backend through APIs, and the backend performs operations on the MySQL database.

For the DevOps part, we use Docker to containerize the application. Docker packages the application and its dependencies into images, and containers are created from these images. Kubernetes is then used to deploy and manage these containers. We can use Minikube to run Kubernetes locally for development and testing.

For CI/CD, we use GitHub Actions. The basic flow is GitHub, then GitHub Actions performs build and testing, creates the Docker image, pushes it to a container registry, and Kubernetes deploys the updated application.

In our literature survey, we studied existing college event-management systems and research related to container orchestration and DevOps. This helped us understand existing approaches and design a system that combines event management with modern containerization and CI/CD practices.

Finally, our architecture consists of the users, React frontend, Node.js and Express backend, backend modules, MySQL database, Docker containers, Kubernetes deployment and the CI/CD pipeline. The main idea is to provide a functional college event management system while also demonstrating practical DevOps concepts.”

One line you absolutely should remember

“React handles the frontend, Node.js and Express handle the backend, MySQL stores the data, Docker containerizes the application, Kubernetes manages the containers, and GitHub Actions automates the CI/CD pipeline.”

Question	Short Answer
What is your project?	A containerized web-based College Event Management System with CI/CD.
Why this project?	To centralize college event management and demonstrate DevOps practices.
Who are the users?	Students and administrators.
Frontend?	React.js.
Backend?	Node.js with Express.js.
Database?	MySQL.
Why MySQL?	It is a relational database suitable for structured users, events and registrations.
What is Docker?	A platform used to package and run applications in containers.
What is a Docker image?	A packaged blueprint containing the application and dependencies.
What is a container?	A running instance of a Docker image.
What is Kubernetes?	A platform for managing and orchestrating containers.
What is a Pod?	The smallest deployable unit in Kubernetes containing one or more containers.
What is Minikube?	A tool for running Kubernetes locally.
What is CI/CD?	Automated building, testing and delivery/deployment of software.
What is GitHub Actions?	A GitHub automation service used to implement our CI/CD workflow.

What you should say

“Good morning everyone. We are Group 11, and our project title is *Design and Development of a Containerized College Event Management System with CI/CD*.

Our project is a centralized web-based platform for managing college events and student registrations.

The main focus of our project is not only event management, but also implementing DevOps concepts. We are using Docker for containerization, Kubernetes for container orchestration, and GitHub Actions for CI/CD automation.”

If teacher asks: What does containerized mean?

Say:

“Containerized means packaging the application and its dependencies into containers so that it can run consistently in different environments.”

Slide 2 — Abstract

This is your **most important slide**, because it explains the complete project.

Your abstract says students can **register/login, view upcoming events, see event details, register for events, and view their registrations**. Administrators can **add, update, delete events and view registered students**.

What you should say

“Our College Event Management System is a web-based application that provides a centralized platform for managing college events and student registrations.

There are mainly two types of users: students and administrators.

Students can register and log in, view upcoming events, check details such as event name, date, time, venue and description, register for events, and view their registrations.

Administrators can manage the complete event lifecycle. They can add new events, update existing events, delete events, and view the students registered for each event.”

Then explain technology

Your slide specifies:

- **React.js** → frontend
- **Node.js + Express.js** → backend
- **MySQL** → database

Say:

“For the frontend, we are using React.js. For the backend, we are using Node.js with Express.js. MySQL is used as our database to store user, event and registration information.

The frontend communicates with the backend through APIs, and the backend performs the required operations on the MySQL database.”

You can explain it like:

“The user interacts with the React frontend. When the user performs an operation, such as registering for an event, React sends an API request to the Node.js and Express backend. The backend processes the request and communicates with MySQL to store or retrieve the required data.”

DevOps Part

This is where your project becomes different from a normal event-management website.

Your slide says the application will be **containerized using Docker**, managed using **Kubernetes**, and a **GitHub Actions CI/CD pipeline** will automate building, testing, creating Docker images and deploying updates.

What you should say

“To implement DevOps concepts, we are containerizing our application using Docker.

Docker packages our application and its dependencies into containers.

Then Kubernetes is used to manage these containers. It handles deployment and container management.

We are also using GitHub Actions for CI/CD. When we make changes to our code and push them to GitHub, the CI/CD pipeline can automatically build and test the application, create the Docker image, and deploy the updated version to Kubernetes.”

Docker Image

“A Docker image is a packaged blueprint of our application containing the application code, dependencies and required environment.”

Container

“A container is a running instance of a Docker image.”

Easy example

“The Docker image is like a blueprint or package, while the container is the running application created from that image.”

🔥 Teacher May Ask: Why Docker?

Say:

“We use Docker to provide a consistent environment for our application. It packages the application and its dependencies together and avoids environment-related problems such as ‘it works on my computer but not on another computer.’”

🔥 Teacher May Ask: Why Kubernetes?

Say:

“Kubernetes is used to manage and orchestrate our containers. It helps with deployment, scaling, networking and maintaining the desired state of the application.”

Very simple:

Docker → creates/runs containers

Kubernetes → manages those containers

🔥 Teacher May Ask: What is a Pod?

Say:

“A Pod is the smallest deployable unit in Kubernetes. It contains one or more containers. In our project, a Pod can run an application container such as our backend or frontend container.”

Teacher May Ask: What is Minikube?

If your architecture mentions Minikube, say:

“Minikube is a tool that allows us to run a Kubernetes cluster locally on our computer. We use it for development, testing and demonstration without requiring a paid cloud platform.”

🔥 Teacher May Ask: What is CI/CD?

Your presentation specifically uses GitHub Actions for CI/CD.

Say:

“CI stands for Continuous Integration. It means automatically building and testing code changes.

CD stands for Continuous Delivery or Continuous Deployment. It automates the process of delivering or deploying the application after the changes are successfully built and tested.”

Literature Survey

We studied existing research related to college event management systems, digital event platforms, container orchestration and DevOps practices.

The literature helped us understand how existing systems handle event management and registration, and how technologies such as Kubernetes and CI/CD can be used to improve application deployment and management.

Based on this study, we designed our system by combining the event-management functionality with modern DevOps practices such as Docker, Kubernetes and CI/CD.”

Why is your project called "Containerized"?

Because you are using **Docker containers** to package the application.

Say:

“It is called containerized because the application components are packaged and run inside Docker containers, making the application portable and consistent.”

★ 2. Why do we need both Docker AND Kubernetes?

This is a very common question.

Docker:

Packages and runs the application in containers.

Kubernetes:

Manages and orchestrates those containers.

Say:

“Docker and Kubernetes have different responsibilities. Docker is used for containerization, whereas Kubernetes is used for managing and orchestrating the containers.”

Remember:

Docker = Package

Kubernetes = Manage

★ 3. What happens if a container fails?

Say:

“Kubernetes can detect failures and recreate the required Pod according to the desired state. This provides self-healing.”

Don't say that Docker itself automatically fixes the application.

★ 4. What happens when you update your project?

This is important because your title contains **CI/CD**.

Say:

“When we make changes to the source code and push them to GitHub, GitHub Actions triggers the CI/CD workflow. It builds and tests the application, creates an updated Docker image, pushes it to the container registry, and the updated image can then be deployed to Kubernetes.”

★ 5. Why GitHub?

“GitHub is used for version control and storing our source code. It also integrates directly with GitHub Actions, which we use for our CI/CD pipeline.”

★ 6. Why GitHub Actions?

“We use GitHub Actions because it provides workflow automation directly within GitHub and can automate our build, test, Docker image creation and deployment process.”

★ 7. What is a Dockerfile?

Very likely question.

“A Dockerfile is a text file containing instructions that tell Docker how to build a Docker image.”

Simple flow:

Dockerfile
↓
docker build
↓
Docker Image
↓
Container

★ 8. What is a Docker image?

“A Docker image is a packaged, read-only template containing the application, dependencies and required environment needed to create a container.”

★ 9. What is a container?

“A container is a running instance of a Docker image in an isolated environment.”

★ 10. What is Kubernetes Service?

If this comes in your architecture/viva:

“A Kubernetes Service provides stable networking and access to Pods. It allows different components of our application to communicate with the required Pods.”

For example:

React Frontend
↓
Backend Service
↓
Backend Pod

★ 11. What is scaling?

“Scaling means increasing or decreasing the number of application instances according to workload.”

Example:

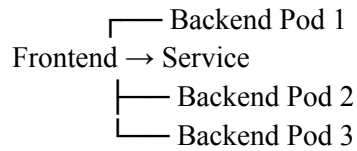
Low traffic → 1 Backend Pod

High traffic → 3 Backend Pods

Kubernetes can manage this.

★ 12. What is load balancing?

If there are multiple backend Pods:



The Service can distribute requests among available Pods.

Say:

“Load balancing distributes incoming requests across available application instances to avoid overloading a single instance.”

★ 13. Why MySQL?

Teacher may ask why you selected MySQL instead of MongoDB.

Say:

“We selected MySQL because our data is structured and relational. Users, events and registrations have clear relationships, so a relational database is suitable for our project.”

★ 14. What are your main database tables?

Remember these three:

Users

Stores:

- User ID
- Name
- Email
- Password
- Role

Events

Stores:

- Event ID
- Event name
- Date
- Time
- Venue
- Description

Registrations

Connects students with events.

User → Registration ← Event

This is a nice point to mention if asked about database relationships.

★ 15. What is an API?

Because your architecture says frontend communicates with backend through APIs.

Say:

“An API is an interface that allows different software components to communicate with each other.”

Example:

```

React
↓
API Request
↓
Express Backend
↓
MySQL
  
```

For example, when a student registers for an event, React sends the registration request to the backend API.

★ 16. What does CRUD mean?

Your admin performs event management, so know this.

CRUD =

- **C** → Create
- **R** → Read

- **U** → Update
- **D** → Delete

Example:

Admin creates an event → Create
Admin views event → Read
Admin changes event details → Update
Admin removes event → Delete

★ 17. What is modular architecture?

Say:

“Modular architecture means dividing the application into separate modules, where each module performs a specific responsibility.”

Our modules:

Authentication
Event Management
Registration
User Management

This makes the system easier to develop, maintain and test.

★ 18. What is the main advantage of your project?

Good answer:

“The main advantage is that our project combines a practical college event management system with DevOps practices. Instead of only developing the application, we demonstrate containerization, orchestration and automated CI/CD.”

This is particularly important because **your project isn't just an event website.**

★ 19. What is innovative/different about your project?

Don't claim that the event-management idea itself is completely new.

Say:

“The event-management functionality already exists in different forms. Our focus is on combining this functionality with a containerized DevOps deployment approach using Docker, Kubernetes and CI/CD.”

That's a much safer answer.

★ 20. What is the complete flow?

If teacher says:

“Explain your whole project in one flow.”

Say:

“First, the student or administrator accesses the React.js frontend. The frontend communicates with the Node.js and Express.js backend through APIs. The backend handles authentication, event management and registrations and stores the data in MySQL. We then containerize the application using Docker. The containers are deployed and managed using Kubernetes. Finally, GitHub Actions automates the CI/CD process by building, testing, creating Docker images and deploying updated versions.”

🔥 MOST IMPORTANT DIFFERENCES TO MEMORIZE

Concept	One-line answer
Docker	Platform for containerizing and running applications
Docker Image	Blueprint/package used to create a container
Container	Running instance of an image
Dockerfile	Instructions for building an image
Pod	Smallest deployable unit in Kubernetes
Kubernetes	Manages and orchestrates containers/Pods
Minikube	Runs Kubernetes locally
Service	Provides stable networking/access to Pods
GitHub	Stores and versions source code
GitHub Actions	Automates CI/CD workflows
CI	Build/test code changes automatically

CD	Deliver/deploy updated application
API	Allows software components to communicate
CRUD	Create, Read, Update, Delete
Scaling	Increasing/decreasing application instances
Self-healing	Kubernetes recreates failed workloads

CRUD — YES

Your project definitely uses CRUD for the **admin event management**:

- **Create** → Add an event
- **Read** → View events
- **Update** → Update event details
- **Delete** → Delete an event

Say:

“Kubernetes can support scaling, but scaling is not a specifically implemented feature in our current project.”